

Disconnected Entities in Entity Framework Core (EF Core)

In this article, I will discuss **Disconnected Entities in Entity Framework Core (EF Core)** with Examples. Please read our previous article, discussing [Asynchronous Programming with Entity Framework Core](#) with Examples.

Type of Entities in Entity Framework Core:

In Entity Framework Core, entities are categorized into two types based on their relationship with the DbContext as follows:

- **Connected Entities:** The DbContext actively tracks these entities. Any changes made to them are automatically detected by the DbContext's change tracker, and the changes are persisted in the database when the **SaveChanges()** method is called. These entities maintain their state (e.g., **Added, Modified, Deleted, or Unchanged**) within the DbContext's change tracker.
- **Disconnected Entities:** These entities are no longer tracked by any DbContext instance. This typically happens when entities are fetched from the database, manipulated outside the scope of the DbContext, and then returned for further processing. Disconnected entities require explicit state management when reattached to another DbContext.

Disconnected Entities Entity Framework Core:

Disconnected entities are not currently tracked by any DbContext instance but were previously associated with one DbContext instance. Since the DbContext is not tracking their state, any modifications made will not be automatically saved to the database.

The primary challenge with disconnected entities is managing their state explicitly when they are reattached to a new DbContext. As a developer, we need to explicitly set the entity state (Added, Modified, Deleted, etc.) correctly to indicate the type of database operation that should be performed.

How to Manage Disconnected Entities in Entity Framework Core:

To handle disconnected entities, the entity must be reattached to the DbContext with the correct entity state to allow Entity Framework Core to track the changes and perform the necessary database operations. We need to explicitly set the entity's state using **context.Entry(entity).State = EntityState.[State]** where **[State]** can be **Added, Modified, or Deleted**. Then, we need to call **SaveChanges()** to persist the changes to the database.

Real-time Example of Disconnected Entities in Entity Framework Core:

Disconnected entities are common in Web APIs, where entities are fetched from the server using one DbContext, modified by the client application, and then returned to the Web API for updating. For example, consider a Web API for managing products:

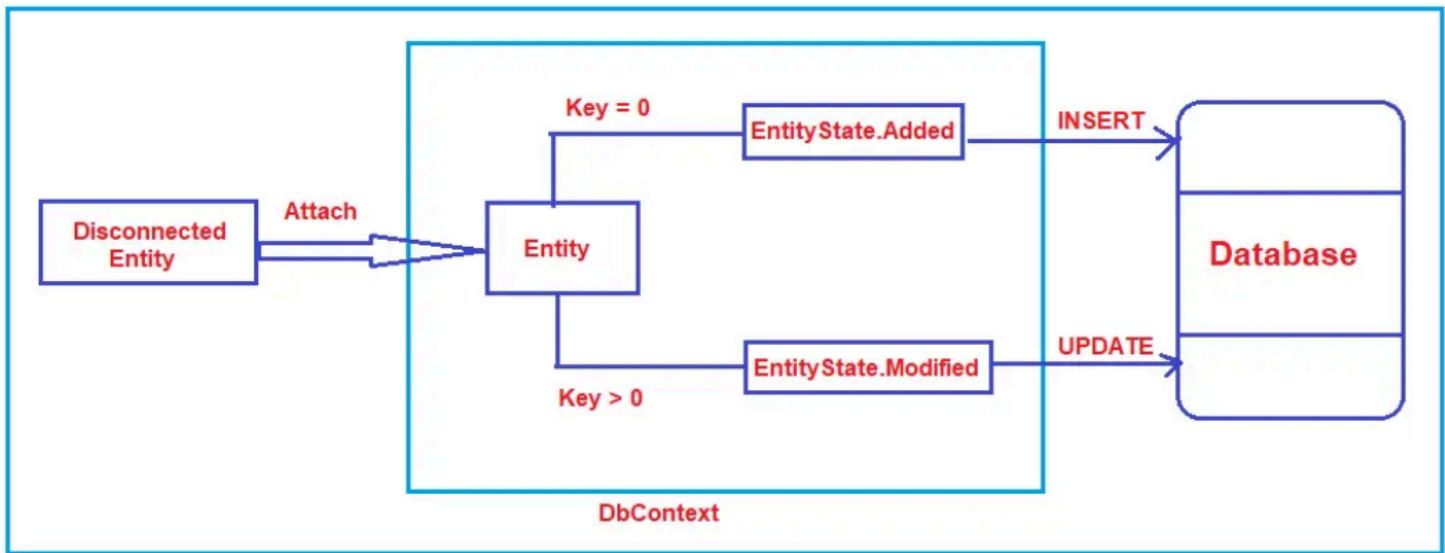
- **Client Requests a Product:** The client sends a request to the server to get product information that needs to be edited.
- **Server Sends the Product:** The server retrieves the product from the database using DbContext and sends it to the client.
- **Client Modifies the Product:** The client modifies the product details on the client side.
- **Client Sends Back the Modified Product:** The client then sends the modified product back to the server.
- **Server Saves Changes:** The server receives the disconnected product entity, reattaches it to the DbContext with a Modified state, and updates the database accordingly by calling the SaveChanges method.

How Do We Save a Disconnected Entity in Entity Framework?

In the Entity Framework Core Disconnected Environment, the context object is not aware of the state of the entities, whether they are new or existing. In this case, first, we need to attach the disconnected entities to the context object with the appropriate Entity State in order to perform INSERT, UPDATE, or DELETE operations in the database.

In the Disconnected Scenario, it is our key responsibility to determine whether the entity is new or existing and set the appropriate entity state based on this. The important question is how to determine whether the entity is new or existing. For this, we need to check the key property value, i.e., the Primary key value of the entity.

If the key property value is greater than zero (in case of Integer column) or Not Null or Empty (in case of String Column), then it is an existing Entity, and we can set the Entity State as Modified. On the other hand, if the key value is zero (in case of Integer column) or Null or Empty (in case of String Column), then it is a new entity, and we need to set the Entity State as Added. For a better understanding, please have a look at the below diagram.



Example to Understand Disconnected Entity in Entity Framework Core:

Let us understand how to Insert, Update, and Delete Entities in the Entity Framework Core Disconnected scenario. For this, we are going to use the following Student Entity.

Student Entity:

```
namespace EFCoreCodeFirstDemo.Entities
{
    public class Student
    {
        public int StudentId { get; set; }
        public string FirstName { get; set; }
        public string LastName { get; set; }
    }
}
```

DbContext Configuration:

Next, modify the EFCoreDbContext class as follows:

```
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Logging;
```

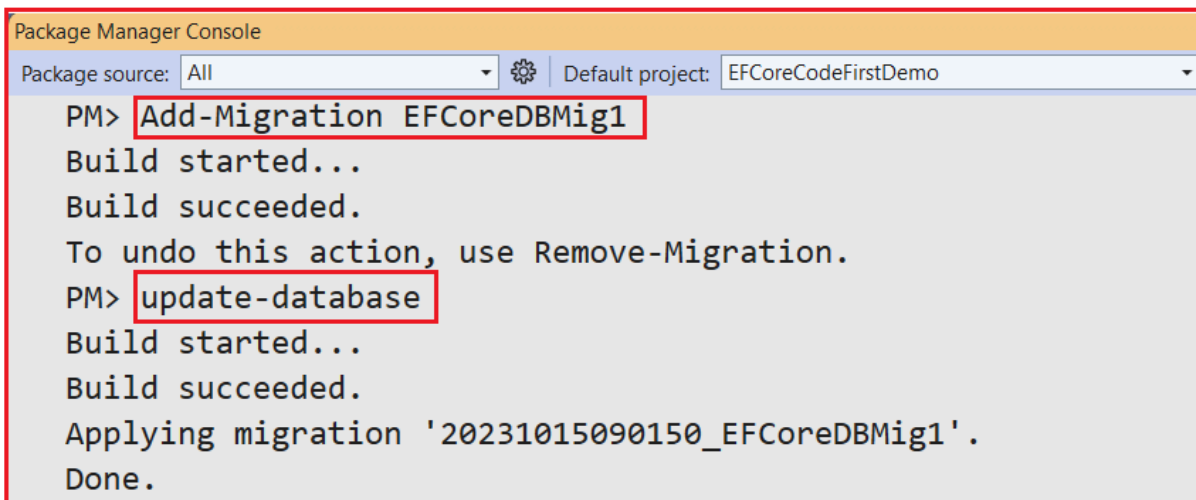
```

namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            //To Display the Generated SQL Statements
            optionsBuilder.LogTo(Console.WriteLine, LogLevel.Information);
            //Configuring the Connection String
            optionsBuilder.UseSqlServer(@"Server=LAPTOP-
6P5NK25R\SQLSERVER2022DEV;Database=EFCoreDB;Trusted_Connection=True;TrustServerCertificate=True");
        }
        public DbSet<Student> Students { get; set; }
    }
}

```

Generating and Applying Migration:

Open the Package Manager Console and Execute the **Add-Migration** and **Update-Database** commands as follows.



```

Package Manager Console
Package source: All [v] [g] Default project: EFCoreCodeFirstDemo [v]
PM> Add-Migration EFCoreDBMig1
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM> update-database
Build started...
Build succeeded.
Applying migration '20231015090150_EFCoreDBMig1'.
Done.

```

How to Save a New Entity in Entity Framework Core Disconnected Scenario:

When inserting a new entity in a disconnected scenario, we need to explicitly attach the entity to the DbContext and set its state to Added. This signals EF Core to generate an INSERT SQL statement. For a better understanding, please modify the Program class as follows. The following example code is self-explained, so please read the comment lines for a better understanding.

```

using EFCoreCodeFirstDemo.Entities;
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo
{
    public class Program
    {
        static void Main(string[] args)
        {
            try
            {

```

```

{
    // Create a new disconnected Student entity
    Student newStudent = new Student()
    {
        FirstName = "Pranaya",
        LastName = "Rout"
    };
    using var context = new EFCoreDbContext();
    // Determine the state based on StudentId
    if (newStudent.StudentId > 0)
    {
        // Existing entity: set state to Modified
        context.Entry(newStudent).State = EntityState.Modified;
    }
    else if (newStudent.StudentId == 0)
    {
        // New entity: set state to Added
        context.Entry(newStudent).State = EntityState.Added;
    }
    else
    {
        throw new Exception("Invalid Student ID");
    }
    // Display the entity state before saving
    Console.WriteLine($"Before SaveChanges - Entity State:
{context.Entry(newStudent).State}\n");
    // Persist changes to the database
    context.SaveChanges();
    // Display the entity state after saving
    Console.WriteLine($"After SaveChanges - Entity State:
{context.Entry(newStudent).State}");
    // Display the Student Id
    Console.WriteLine($"Student ID: {newStudent.StudentId}");
    Console.ReadLine();
}
catch (Exception ex)
{
    Console.WriteLine($"Error: {ex.Message}");
}
}
}
}

```

Explanation:

- Since StudentId is 0, the entity is marked as Added.
- Upon calling SaveChanges(), EF Core generates and executes an INSERT statement.
- The StudentId is updated with the newly generated identity value from the database.
- After saving, the entity state changes to Unchanged.

Run the above code, and you should get the following output.

Before SaveChanges - Entity State: Added

```
info: 21-09-2024 20:29:32.051 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database.Command)
      Executed DbCommand (63ms) [Parameters=[@p0='?' (Size = 4000), @p1='?' (Size = 4000)], CommandType='Text', CommandTimeout='30']
      SET IMPLICIT_TRANSACTIONS OFF;
      SET NOCOUNT ON;
      INSERT INTO [Students] ([FirstName], [LastName])
      OUTPUT INSERTED.[StudentId]
      VALUES (@p0, @p1);
```

After SaveChanges - Entity State: Unchanged
Student ID: 1

How to Update an Existing Entity in Entity Framework Core Disconnected Scenario

The following is an example of the Entity Framework Core Disconnected Scenario for updating an Existing Entity. In the example below, we set the value of the StudentId property, and hence, it will assign the entity State to the Modified state. When we run the following application, it will generate and execute the UPDATE SQL Statement. Please ensure the StudentId specified here in the Student entity exists in the database; otherwise, you will get an exception.

```
using EFCoreCodeFirstDemo.Entities;
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo
{
    public class Program
    {
        static void Main(string[] args)
        {
            try
            {
                // Create a disconnected Student entity with an existing
                StudentId
                Student existingStudent = new Student()
                {
                    StudentId = 1, // Ensure this ID exists in the database
                    FirstName = "Pranaya",
                    LastName = "Rout Updated"
                };
                using var context = new EFCoreDbContext();
                // Determine the state based on StudentId
                if (existingStudent.StudentId > 0)
                {
                    // Existing entity: set state to Modified
                    context.Entry(existingStudent).State =
                    EntityState.Modified;
                }
                else if (existingStudent.StudentId == 0)
                {
                    // New entity: set state to Added
                    context.Entry(existingStudent).State = EntityState.Added;
                }
            }
            else
            {
                // Exception handling
            }
        }
    }
}
```

```

        throw new Exception("Invalid Student ID");
    }
    // Display the entity state before saving
    Console.WriteLine($"Before SaveChanges - Entity State:
{context.Entry(existingStudent).State}\n");
    // Persist changes to the database
    context.SaveChanges();
    // Display the entity state after saving
    Console.WriteLine($"
\nAfter SaveChanges - Entity State:
{context.Entry(existingStudent).State}");
    Console.ReadLine();
}
catch (Exception ex)
{
    Console.WriteLine($"Error: {ex.Message}");
}
}
}
}

```

Explanation:

- Since StudentId is greater than 0, the entity is marked as Modified.
- Upon calling SaveChanges(), EF Core generates and executes an UPDATE statement.
- After saving, the entity state changes to Unchanged.

Run the above code, and you should get the following output.

```

Before SaveChanges - Entity State: Modified

info: 21-09-2024 20:34:28.340 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.
Database.Command)
    Executed DbCommand (32ms) [Parameters=[@p2='?' (DbType = Int32), @p0='?' (Size = 4000), @p1='?'
(Size = 4000)], CommandType='Text', CommandTimeout='30']
    SET IMPLICIT_TRANSACTIONS OFF;
    SET NOCOUNT ON;
    UPDATE [Students] SET [FirstName] = @p0, [LastName] = @p1
    OUTPUT 1
    WHERE [StudentId] = @p2;

After SaveChanges - Entity State: Unchanged

```

How to Delete a Disconnected Entity in Entity Framework Core:

Deleting an entity involves setting its state to Deleted and ensuring that the primary key exists in the database. For a better understanding, please modify the Program class as follows. In the below example, the student object contains only the StudentId key property. Deleting an entity using the entity framework only requires the key property.

```

using EFCoreCodeFirstDemo.Entities;
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo
{
    public class Program
    {

```

```

static void Main(string[] args)
{
    try
    {
        // Create a student object with an existing StudentId
        (disconnected entity)
        Student student = new Student()
        {
            StudentId = 1 // Assume this ID exists in the database
        };
        using var context = new EFCoreDbContext();
        // Set the entity state to Deleted
        context.Entry(student).State = EntityState.Deleted;
        // Display the entity state before saving
        Console.WriteLine($"Before SaveChanges - Entity State:
{context.Entry(student).State}\n");
        // Persist changes to the database
        context.SaveChanges();
        // Display the entity state after saving
        Console.WriteLine($"After SaveChanges - Entity State:
{context.Entry(student).State}");
        Console.ReadLine();
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Error: {ex.Message}");
    }
}
}
}

```

Explanation:

- The entity is marked as Deleted.
- Upon calling SaveChanges(), EF Core generates and executes a DELETE statement.
- After deletion, the entity state changes to Detached since it's no longer tracked.

So, when you run the above example, you will get the following output.

```

Before SaveChanges - Entity State: Deleted

info: 21-09-2024 20:39:01.144 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database.Command)
      Executed DbCommand (32ms) [Parameters=[@p0='?' (DbType = Int32)], CommandType='Text', CommandTim
eout='30']
      SET IMPLICIT_TRANSACTIONS OFF;
      SET NOCOUNT ON;
      DELETE FROM [Students]
      OUTPUT 1
      WHERE [StudentId] = @p0;

After SaveChanges - Entity State: Detached

```

When Should We Use Disconnected Entity in Entity Framework Core?

The following are some of the common scenarios where disconnected entities are beneficial in Entity Framework core:

- **Web Applications and APIs:** HTTP is stateless, meaning each request is independent. Entities are often sent to and received from clients in a disconnected state.
- **Desktop Applications:** Where entities are modified and then saved in a separate context.
- **Batch Processing:** Processing large datasets in batches, where DbContext instances are created and disposed of within each batch, benefits from disconnected entity management.

In the next article, I will discuss [Batch Processing with a Job Scheduler](#) with Examples. In this article, I explain the **Disconnected Entities in Entity Framework Core** with Examples. I hope you enjoyed this article on Disconnected Entities in EF Core.